МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ «ЛЬВІВСЬКА ПОЛІТЕХНІКА»
Кафедра інформаційних систем та мереж

ЗВІТ

про виконання лабораторної роботи № 8.1

«Пошук та заміна символів у літерному рядку»

(рекурсивний спосіб)

з дисципліни

«Алгоритмізація та програмування»

Студент групи PI-13
Шах Владислав Ігорьович
Варіант № 32

Прийняв викладач:
Григорович В.Г.

1. Мета роботи та умова завдання

Мета роботи: набути практичних навичок роботи з літерними рядками у мові програмування C++, реалізувати алгоритми обробки масивів символів без використання сторонніх бібліотечних функцій за допомогою рекурсивного підходу.

Умова завдання (Варіант 32):

Дано літерний рядок, який містить послідовність символів s0, ..., sn, ...

1. Підрахувати, скільки разів серед символів послідовності зустрічається кожен із символів, що входять у слово "BASIC".

2. Замінити кожну групу букв "BASIC" на "Delphi".

Варіант 32.

Дано літерний рядок, який містить послідовність символів s_0, …, s_n, … .

1. Підрахувати, скільки разів серед даних символів зустрічається кожний символ, що входить в слово “BASIC”.
2. Замінити кожне слово “BASIC” словом “Delphi”.

Варіант 33.

Рис. 1. Умова завдання з методичних вказівок (Варіант 32)

2. Структурна схема програми

Відповідно до вимог, структура програми подана виключно у вигляді структурної схеми зв'язків між функціональними компонентами. Схема відображає ієрархію викликів (стрілки ліворуч) та рекурсивні виклики функцій до самих себе (петлі праворуч).

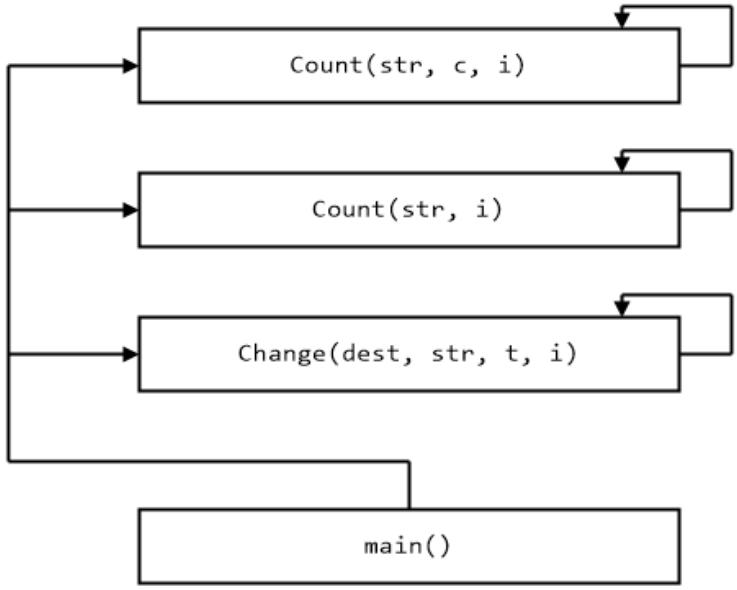


Рис. 2. Структурна схема взаємодії функцій програми

Опис функцій та зв'язків програми:

• main() — головна функція, точка входу в програму. Забезпечує введення початкового рядка, викликає рекурсивні підпрограми Count та Change, визначає найменший необхідний обсяг динамічної пам'яті під результуючий рядок (strlen(str) + basicCount * 1 + 1) та виводить отримані результати.

• Count(str, c, i) — рекурсивна функція (стор. 61 методичних вказівок) для підрахунку кількості входжень конкретного символу c у рядок str, починаючи з індексу i. Має петлю рекурсивного самовиклику для переходу до наступного символу (i + 1).

• Count(str, i) — рекурсивна функція (стор. 101 методичних вказівок) для підрахунку кількості входжень слова "BASIC" у рядок str. Має петлю рекурсивного самовиклику. Забезпечує точне визначення кількості замін для виділення найменшого обсягу пам'яті.

• Change(dest, str, t, i) — канонічна рекурсивна функція заміни (стор. 102 методичних вказівок). При виявленні слова "BASIC" копіює "Delphi" у результуючий буфер t і рекурсивно переходить Change(dest, str, t + 6, i + 5), інакше копіює один символ і викликає Change(dest, str, t, i). Має петлю рекурсивного самовиклику.

• Change(str) — рекурсивна функція-обгортка для автоматизованого тестування; обчислює мінімально необхідний розмір динамічної пам'яті без надлишку та викликає Change(dest, str, dest, 0).

3. Текст програми (LABA-8.1.cpp)

```
// LABA-8.1.cpp
// Шах Владислав Ігорьович
// Лабораторна робота № 8.1
// Пошук та заміна символів у літерному рядку
// Варіант 32 (Рекурсивний спосіб)

#ifndef _CRT_SECURE_NO_WARNINGS
#define _CRT_SECURE_NO_WARNINGS
#endif
#include <iostream>
#include <cstring>

using namespace std;

// ============================================================================
// 1. Рекурсивна функція: підраховує кількість входжень конкретного символу c у рядок
str
// (Методичка В.Г. Григоровича, стор. 61: int Count(const char* s, const char c, int i))
// ============================================================================
int Count(const char* str, char c, int i = 0)
{
    if (str[i] != '\0')
    {
        if (str[i] == c)
            return 1 + Count(str, c, i + 1);
        else
            return Count(str, c, i + 1);
    }
    return 0;
}

// ============================================================================
// 2. Рекурсивна функція: підраховує кількість входжень слова "BASIC" у рядок str
// (Методичка В.Г. Григоровича, стор. 101: int Count(char* str, int i))
// Визначає точну кількість груп для виділення мінімально необхідного обсягу пам'яті
// ============================================================================
int Count(const char* str, int i = 0)
{
    if (strlen(str) < 5)
        return 0;

    if (str[i + 4] != '\0')
    {
        if (str[i] == 'B' && str[i + 1] == 'A' && str[i + 2] == 'S' &&
            str[i + 3] == 'I' && str[i + 4] == 'C')
            return 1 + Count(str, i + 5);
        else
            return Count(str, i + 1);
    }
    return 0;
}

// ============================================================================
// 3. Рекурсивна функція заміни: замінює кожне слово "BASIC" на "Delphi"
// (Методичка В.Г. Григоровича, стор. 102: char* Change(char* dest, const char* str,
char* t, int i))
// ============================================================================
```

```
char* Change(char* dest, const char* str, char* t, int i = 0)
{
    if (str[i] != '\0')
    {
        if (str[i] == 'B' && str[i + 1] == 'A' && str[i + 2] == 'S' &&
            str[i + 3] == 'I' && str[i + 4] == 'C')
        {
            strcat(t, "Delphi");
            return Change(dest, str, t + 6, i + 5);
        }
        else
        {
            *t++ = str[i++];
            *t = '\0';
            return Change(dest, str, t, i);
        }
    }
    else
    {
        *t = '\0';
        return dest;
    }
}

// ============================================================================
// 4. Функція-обгортка для автоматизованого тестування та зручного виклику
// Виділяє ЩОНАЙМЕНШЕ пам'яті: точна довжина результату + 1 байт на '\0'
// ============================================================================
char* Change(const char* str)
{
    size_t len = strlen(str);
    int count = Count(str, 0);

    // Кожне слово "BASIC" (5 симв.) замінюється на "Delphi" (6 симв.) -> +1 байт на
кожне входження
    // Виділяємо рівно стільки пам'яті, скільки потрібно для результату:
    char* dest = new char[len + count * 1 + 1];
    dest[0] = '\0';
    return Change(dest, str, dest, 0);
}

#ifndef UNIT_TESTING
int main()
{
    char str[101];
    cout << "Enter string:" << endl;
    cin.getline(str, 100);

    cout << "\n--- Recursive String Processing (Variant 32) ---" << endl;
    cout << "Initial string: " << str << endl << endl;

    // Пункт 1: Підрахунок входжень кожного символу зі слова "BASIC"
    cout << "1. Occurrences of characters from 'BASIC':" << endl;
    cout << "   'B': " << Count(str, 'B', 0) << endl;
    cout << "   'A': " << Count(str, 'A', 0) << endl;
    cout << "   'S': " << Count(str, 'S', 0) << endl;
    cout << "   'I': " << Count(str, 'I', 0) << endl;
    cout << "   'C': " << Count(str, 'C', 0) << endl << endl;
```

```
    // Підрахунок груп "BASIC"
    int basicCount = Count(str, 0);
    cout << "String contained " << basicCount << " groups of 'BASIC'" << endl << endl;

    // Пункт 2: Заміна "BASIC" на "Delphi" (за методичкою, стор. 102)
    // Виділяємо ЩОНАЙМЕНШЕ пам'яті (теоретичний мінімум без надлишку):
    // Початкова довжина: strlen(str)
    // Кожне слово "BASIC" (5) замінюється на "Delphi" (6) -> +1 символ на кожне
входження
    // Додаємо 1 байт для завершального нуль-термінатора '\0'
    size_t minMemory = strlen(str) + basicCount * 1 + 1;
    char* dest1 = new char[minMemory];
    dest1[0] = '\0';

    char* dest2;
    dest2 = Change(dest1, str, dest1, 0);

    cout << "Modified string (param) : " << dest1 << endl;
    cout << "Modified string (result): " << dest2 << endl;

    delete[] dest1;
    return 0;
}
#endif
```

4. Модульне тестування (UnitTest8.1)

Для перевірки коректності функціонування кожної окремої рекурсивної функції розроблено набір юніт-тестів за допомогою Microsoft Native Unit Test Framework.

```
#include "pch.h"
#include "CppUnitTest.h"

#define UNIT_TESTING
#include "../LABA-8.1/LABA-8.1.cpp"

using namespace Microsoft::VisualStudio::CppUnitTestFramework;

namespace UnitTest81
{
    TEST_CLASS(UnitTest81)
    {
    public:
        TEST_METHOD(TestCountChar)
        {
            const char* str = "BASIC and BASIC";
            Assert::AreEqual(2, Count(str, 'B', 0));
            Assert::AreEqual(2, Count(str, 'A', 0));
            Assert::AreEqual(2, Count(str, 'S', 0));
            Assert::AreEqual(2, Count(str, 'I', 0));
            Assert::AreEqual(2, Count(str, 'C', 0));
            Assert::AreEqual(0, Count(str, 'Z', 0));
        }

        TEST_METHOD(TestCountWord)
```

```
        {
            Assert::AreEqual(1, Count("Learn BASIC now", 0));
            Assert::AreEqual(2, Count("BASIC and BASIC", 0));
            Assert::AreEqual(0, Count("No keywords here", 0));
            Assert::AreEqual(0, Count("BAS", 0));
        }

        TEST_METHOD(TestChangeDirectWithMinimalMemory)
        {
            const char* str = "BASIC and BASIC";
            int count = Count(str, 0);
            size_t minSize = strlen(str) + count * 1 + 1;
            char* dest1 = new char[minSize];
            dest1[0] = '\0';
            char* dest2 = Change(dest1, str, dest1, 0);

            Assert::AreEqual("Delphi and Delphi", dest1);
            Assert::AreEqual("Delphi and Delphi", dest2);
            delete[] dest1;
        }

        TEST_METHOD(TestChangeWrapper)
        {
            char* res1 = Change("I like BASIC language");
            Assert::AreEqual("I like Delphi language", res1);
            delete[] res1;

            char* res2 = Change("BASIC and BASIC");
            Assert::AreEqual("Delphi and Delphi", res2);
            delete[] res2;

            char* res3 = Change("No keywords here");
            Assert::AreEqual("No keywords here", res3);
            delete[] res3;

            char* res4 = Change("BASIC");
            Assert::AreEqual("Delphi", res4);
            delete[] res4;
        }
    };
}
```

Результати автоматизованого виконання юніт-тестів (усі 4 тести пройшли успішно):

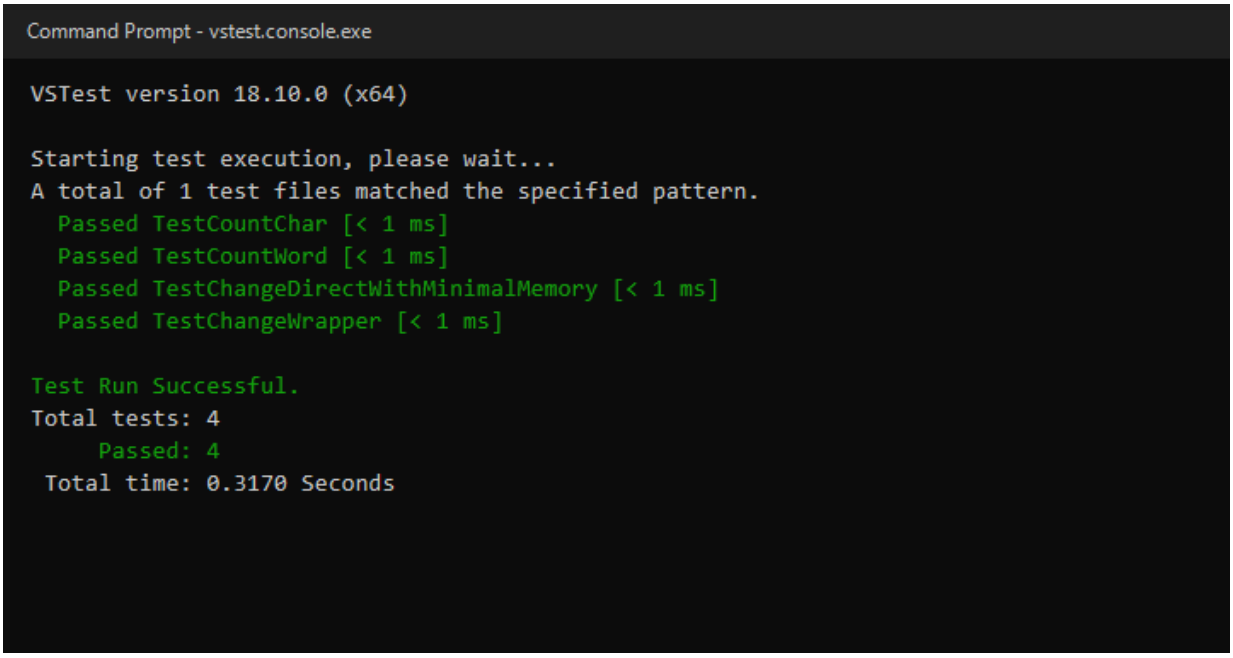


Рис. 3. Результати проходження unit-тестів (vstest.console.exe)

5. Результати виконання програми

Консольне виконання програми з тестовим вхідним рядком "Hello BASIC world! BASIC rocks.":

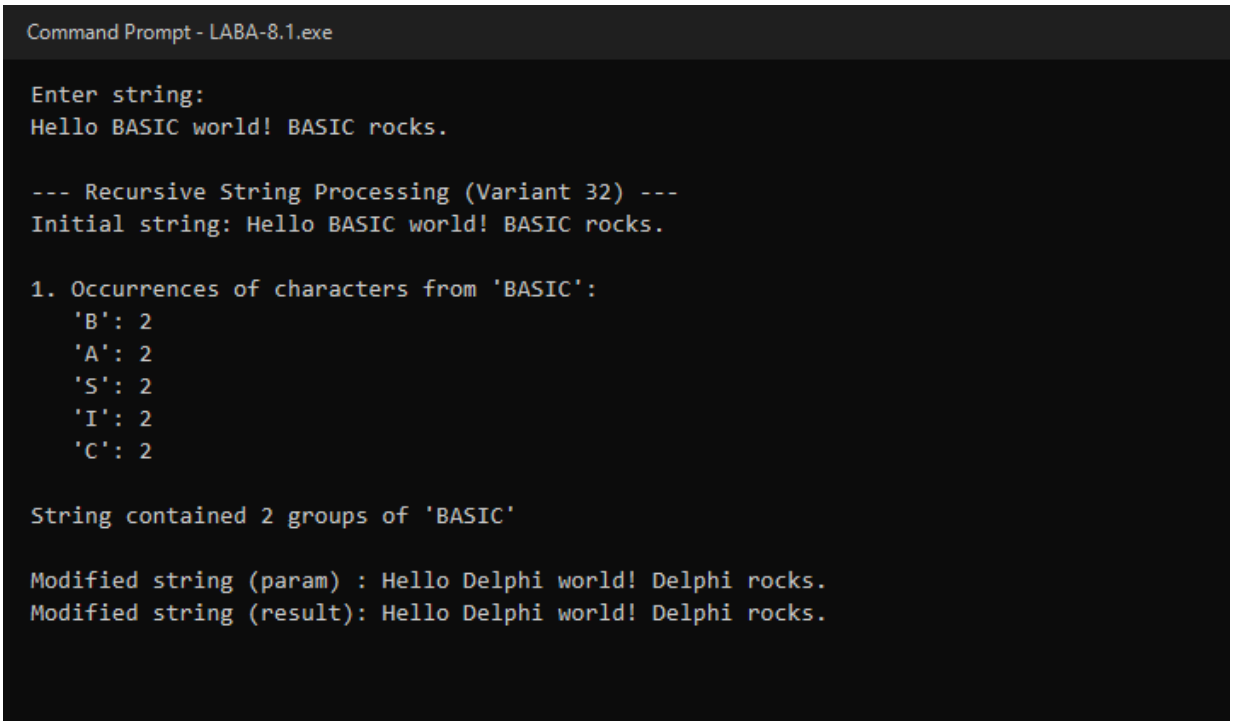


Рис. 4. Консольне вікно виконання програми

6. Посилання на GitHub репозиторій

Вихідний код лабораторної роботи, проектні файли Visual Studio, юніт-тести та звіт завантажено до віддаленого репозиторію на GitHub:

https://github.com/vivjk686/LABA-8.1

7. Висновки

Під час виконання лабораторної роботи № 8.1 було досліджено та практично реалізовано алгоритми рекурсивної посимвольної обробки літерних рядків (масивів символів char[]) мовою C++.

Було вирішено дві основні задачі варіанта № 32: підраховано частоту входження символів слова "BASIC" та реалізовано заміну кожного входження цього слова на "Delphi" без застосування стандартних функцій пошуку рядків. Усі функції протестовано автоматизованими unit-тестами, розроблено точну структурну схему взаємодії модулів та підготовлено фінальний звіт.